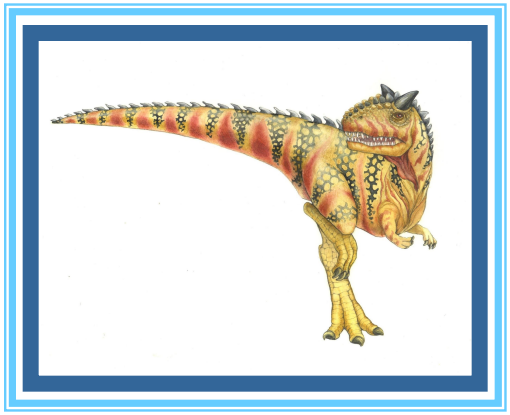
UNIT III: Interprocess Communication and Deadlock



Silberschatz, Galvin and Gagne ©2018

Course Syllabus

UNIT III: Interprocess Communication and Deadlock (09 Hours)

Concurrency: Critical section problem, Synchronization primitives (Semaphores, Mutexes, Monitors) Synchronization Problems: Producer-Consumer, Reader Writer, Dining Philosophers Inter-Process

Communication (IPC): Message passing, Shared memory Deadlocks: Conditions, Prevention, Avoidance (Banker's Algorithm), Detection, Recovery

Case Studies - 1) Interprocess Communication (IPC) in a Banking System 2) Deadlock in Railway Scheduling Systems

Introduction to Syllabus

Course :- PCC-203- COM: Operating Systems

Teaching /scheme	Credits	Examination Scheme
Theory : 03 Hours/Week	03	CCE : 30 Marks End-Semester: 70 Marks

Prerequisite Courses : Data Structure, Digital Electronics

Companion Course : Computer Organization and Microprocessors

Course Objective

1. To **understand** the fundamental concepts, types, and structures of Operating Systems.

2. To **understand** and **analyze** process management concepts.

3. To **identify** and **solve** concurrency and deadlock in the operating system.

4. **Explore** the various techniques of memory management.

5. **Understand** I/O management, disk scheduling, and file systems

Course Outcome

- CO1: **Analyze** the fundamentals of Operating Systems, including types, structures, system calls, and basic Linux commands.

- CO2: **Apply** process scheduling and synchronization to optimize CPU utilization in modern operating systems.

- CO3: **Identify** the mechanism for dealing with deadlocks and concurrency concerns.

- CO4: **Apply** techniques of memory management to solve memory management problems.

- CO5: **Illustrate** I/O and file management policies.

Critical Section in Synchronization

- A critical section is a segment of a program where shared resources, such as memory, files, or ports, are accessed by multiple processes or threads.
- To prevent issues like data inconsistency and race conditions, synchronization techniques ensure that only one process or thread accesses the critical section at a time.
- The critical section includes operations on shared variables or resources that must be executed atomically to maintain data consistency.
- In concurrent programming, if one process modifies shared data while another reads it simultaneously, the outcome can be unpredictable. Therefore, access to shared resources must be synchronized to ensure correct program behavior.

Critical Section in Synchronization

1. **Entry Section**

   The process requests permission to enter the critical section.

   Synchronization tools (e.g., mutex, semaphore) are used to control access.

2. **Critical Section**

   The actual code where shared resources are accessed or modified.

3. **Exit Section**

   The process releases the lock or semaphore, allowing other processes to enter the critical section.

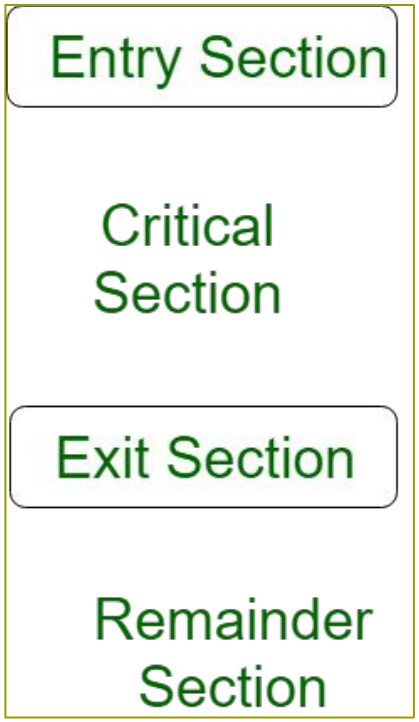
4. **Remainder Section**

   The rest of the program that does not involve shared resource access.

Critical Section

- General structure of process P_i

```
while (true) {
        entry section
            critical section
        exit section
            remainder section
}
```



Characteristics Of Critical Section

1. Mutual Exclusion

- Only one process or thread can execute in the critical section at a given time.
- If two or more processes access shared resources (like variables or files) at the same time without control, data inconsistency or corruption may occur.
- If process Pi is executing in its critical section, then no other processes can be executing in their critical sections.

Example:

- If two threads update the same bank account balance at the same time, the final result may be incorrect due to race conditions.
- We can achieve it by : Using synchronization tools like mutexes, locks, or semaphores to ensure exclusive access.

Characteristics Of Critical Section

2. Progress

- If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.
- It's important to avoid idle CPU cycles or deadlock situations where all processes are waiting, even when they don't have to enter.

Example:

- If process A finishes its work and leaves the critical section, process B (waiting to enter) should not be delayed unnecessarily due to faulty design or logic in the algorithm.
- Goal: To ensure that the system continues to make progress and doesn't freeze or hang.

Characteristics Of Critical Section

3. Bounded Waiting

- There must be a limit on how many times other processes are allowed to enter the critical section before a waiting process gets its turn. It is important to prevent starvation, where one process waits indefinitely while others repeatedly enter the critical section.

Example:

- If process A is always skipped in favor of processes B and C, then A might never enter, even if it's ready.
- Solution: Implement fair scheduling policies like FIFO queues or ticket-based systems.

Handling Critical Section

- Two general approaches are used to handle critical sections

1. Preemptive Kernels : A preemptive kernel allows the operating system to interrupt or preempt a process even when it is running in kernel mode.

- The OS can forcibly switch from one running process to another.
- Even if a process is performing a system call or executing kernel code, it can be paused, and another process can be scheduled.

1. Preemptive Kernels :

- **Advantages:**
- Better responsiveness, especially for real-time systems.
- Higher CPU utilization and fairness among processes.

- **Disadvantages:**
- Increased complexity due to the need to manage race conditions and data consistency when kernel data structures are accessed by multiple processes.

- Example Use Case: Modern desktop and server operating systems like Linux, Windows, and macOS use preemptive kernels for better multitasking.

Handling Critical Section

2. Non-Preemptive Kernels : A non-preemptive kernel does not allow interruption of a process that is running in kernel mode. The CPU control is explicitly released by the process. The kernel ensures that only one process is active in the kernel at any given time.

The process continues until it:

Exits the kernel,

Blocks (e.g., waits for I/O), or

Voluntarily yields the CPU.

Handling Critical Section

Advantages:

- Simplicity: Easier to program and maintain.
- No race conditions on kernel data since access is automatically serialized.

Disadvantages:

- Poor responsiveness, especially if a long-running kernel operation delays other processes.
- Not suitable for real-time or interactive systems.
- Example Use Case: Older operating systems or embedded systems where simplicity and reliability outweigh responsiveness.

Critical Section Problem

The use of critical sections in a program can cause a number of issues, including:

- **Deadlock:**
- **Starvation:**
- **Overhead:**

Critical Section Problem

1. **Deadlock:**

- When two or more threads or processes wait for each other to release a critical section, it can result in a deadlock situation in which none of the threads or processes can move.

- Deadlocks can be difficult to detect and resolve, and they can have a significant impact on a program's performance and reliability.

Critical Section Problem

3. Overhead:

- When using critical sections, threads or processes must acquire and release locks or semaphores, which can take time and resources. This may reduce the program's overall performance.

  It could be visualized using the pseudo-code below

```
do{
    flag=1;
    while(flag); // (entry section)
        // critical section
    if (!flag)
        // remainder section
} while(true);
```

Solution to Critical Section Problem :

A simple solution to the critical section can be thought of as shown below,

```
acquireLock();
Process Critical Section
releaseLock();
```

- A thread must acquire a lock prior to executing a critical section.
- The lock can be acquired by only one thread.
- There are various ways to implement locks in the above pseudo-code.

Examples of critical sections in real-world applications

1. **Banking System (ATM or Online Banking)**
   - **Critical Section:** Updating an account balance during a deposit or withdrawal.
   - **Issue if not handled:** Two simultaneous withdrawals could result in an incorrect final balance due to race conditions.
2. **Ticket Booking System (Airlines, Movies, Trains)**
   - **Critical Section:** Reserving the last available seat.
   - **Issue if not handled:** Two users may be shown the same available seat and both may book it, leading to overbooking.

Examples of critical sections in real-world applications

3. **Print Spooler in a Networked Printer**

- **Critical Section:** Sending print jobs to the printer queue.
- **Issue if not handled:** Print jobs may get mixed up or skipped if multiple users send jobs simultaneously.

4. **File Editing in Shared Documents (e.g., Google Docs, MS Word with shared access)**

- **Critical Section:** Saving or writing to the shared document.
- **Issue if not handled:** Simultaneous edits could lead to conflicting versions or data loss.

Examples of critical sections in real-world applications

5. **Online Multiplayer Gaming Servers**

   Critical Section: Updating a player's score, health, or game state in real time.

   Issue if not handled: Game logic becomes inconsistent; players may see outdated or incorrect data.

6. **Inventory Management in E-Commerce**

   Critical Section: Reducing stock quantity when a product is purchased.

   Issue if not handled: Overselling of items, customer dissatisfaction.

Synchronization primitives

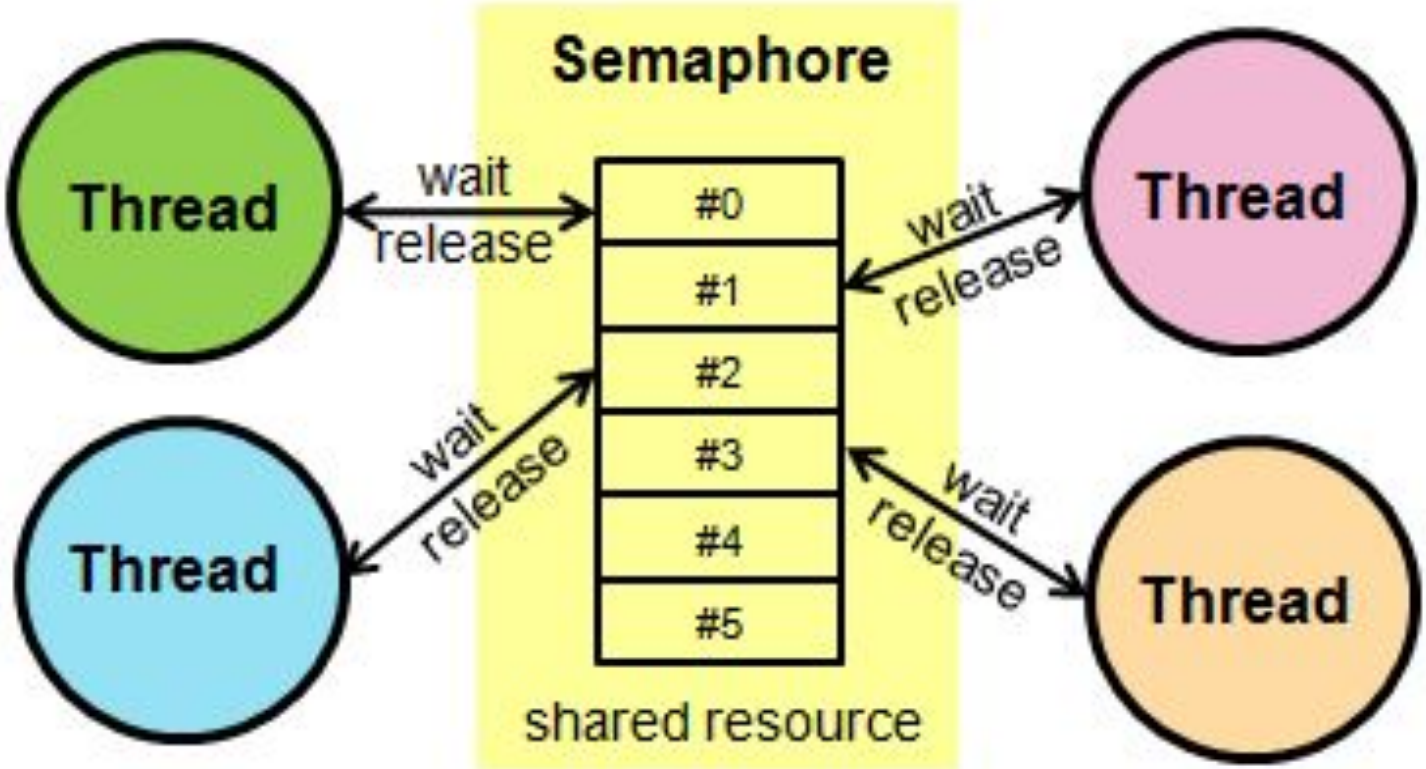
1. **Semaphores**
2. **Mutexes**
3. **Monitors**

- Semaphores, mutexes, and monitors are synchronization mechanisms used in concurrent programming to manage access to shared resources and prevent race conditions.
- Mutexes and semaphores are fundamental tools, while monitors offer a higher-level, structured approach.

1. Semaphores

- A semaphore is a non-negative integer variable that is shared between various threads.
- Semaphore works upon signaling mechanism, in this a thread can be signaled by another thread.
- It provides a less restrictive control mechanism.
- Any thread can invoke signal() (also known as release() or up()), and any other thread can invoke wait() (also known as acquire() or down()).
- There is no strict ownership in semaphores, meaning the thread that signals doesn't necessarily have to be the same one that waited. Semaphores are often used for coordinating signaling between threads. Semaphore uses two atomic operations for process synchronisation:
- Wait (P)
- Signal (V)

1. Semaphores



1. Semaphores

- A Semaphore is a lower-level object.
- If a thread tries to acquire the semaphore then it is allowed, and the counter is decremented.
- When a thread is done then it releases the semaphore, and increments the counter.
- If the counter is already zero when a thread tries to acquire the semaphore then it has to wait until another thread releases the semaphore.
- If multiple threads are waiting when a thread releases a semaphore then one of them gets it.
- The thread that releases a semaphore need not be the same thread that acquired it.

Types of Semaphores

Semaphores are of two Types:

1. **Binary Semaphore:** This is also known as a mutex lock, as they are locks that provide mutual exclusion. It can have only two values - 0 and 1. Its value is initialized to 1. It is used to implement the solution of critical section problems with multiple processes and a single resource.

2. **Counting Semaphore:** Counting semaphores can be used to control access to a given resource consisting of a finite number of instances. The semaphore is initialized to the number of resources available. Its value can range over an unrestricted domain.

2. Monitor

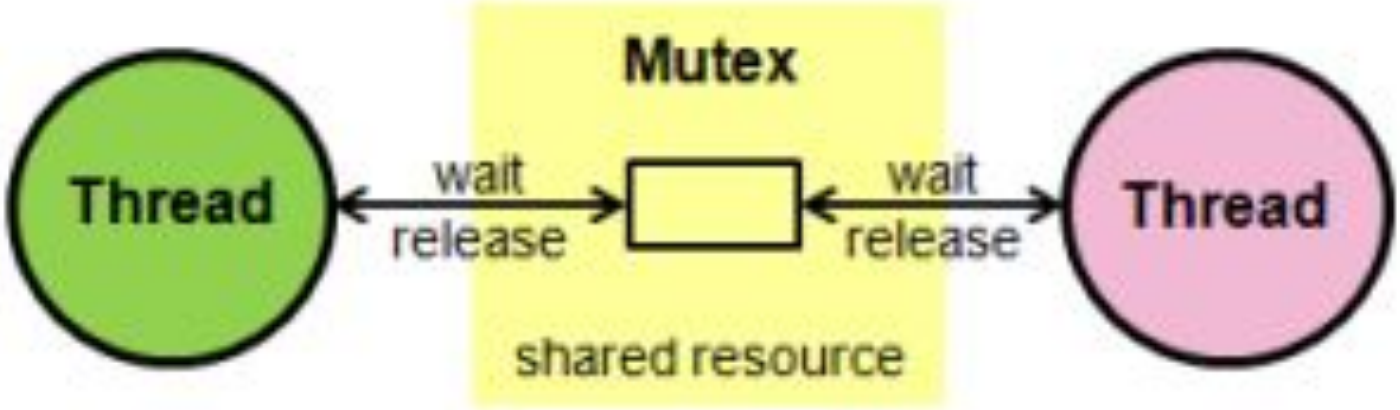
- Controls only one thread at a time , and can execute in the monitor.
- A monitor is a set of multiple routines, shared data encapsulated as a single unit like object.
- At a time only one thread can be enter into it and control by a guard.
- Monitors are simpler to use than semaphores because they handle all of the details of lock acquisition and release.
- Use semaphore to implement a monitor.
- Operations are wait() & signal().

3. Mutex

- Used to provide mutual exclusion.
- A mutex can be used to synchronize threads across processes.
- When used for interprocess synchronization, a mutex is called a named mutex because it is to be used in another application, and therefore it cannot be shared by means of a global or static variable.
- Operations are wait() & release().
- It is a simple lock,when someone is holding this lock no others are permitted to access what is protected by the lock until the holds releases it.

3. Mutex

- The limitation of mutex is that it fails to distinguish between different kind of accesses.ie, those who modify the protected data(called write) and those who donot modify the data(called read),multiple reads at the same time should be allowed.
- Read-Write locks to address this kind of situation.



Difference Between Mutex and Semaphore

Mutex	Semaphore
A mutex is an object.	A semaphore is an integer.
Mutex works upon the locking mechanism.	Semaphore uses signaling mechanism.
Operations on mutex: • Lock • Unlock	Operation on semaphore: • Wait • Signal
Mutex does not have any subtypes.	Semaphore is of two types: • Counting Semaphore • Binary Semaphore

Difference Between Mutex and Semaphore

Mutex	Semaphore
A mutex can only be modified by the process that is requesting or releasing a resource.	Semaphore work with two atomic operations (Wait, signal) which can modify it.
If the mutex is locked then the process needs to wait in the process queue, and mutex can only be accessed once the lock is released.	If the process needs a resource, and no resource is free. So, the process needs to perform a wait operation until the semaphore value is greater than zero.

Synchronization Problems OR Classical IPC Problems

- Inter-Process Communication (IPC) is necessary for processes to communicate and share data.
- While basic communication between processes may sound simple, certain situations can cause issues that need specific solutions.
- These situations are known as Classical IPC Problems, which involve managing synchronization, avoiding deadlock, and ensuring that resources are accessed in a controlled manner.
- Some of the well-known Inter Process Communication problems are:

1. **Producer-Consumer**
2. **Reader Writer**
3. **Dining Philosophers Inter-Process**

1. Producer Consumer Problem

- The Producer-Consumer Problem involves two types of processes: the Producer, which creates data, and the Consumer, which processes that data.
- The challenge is ensuring that the Producer doesn't overfill the buffer, and the Consumer doesn't try to consume data from an empty buffer.

Key Problems in the Producer-Consumer Problem:

1. **Buffer Overflow**: If the producer tries to add data when the buffer is full, there will be no space for new data, causing the producer to be blocked.
2. **Buffer Underflow**: If the consumer tries to consume data when the buffer is empty, it has nothing to consume, causing the consumer to be blocked.

Solution to Producer Consumer Problem

This problem can be solved using synchronization techniques such as semaphores or mutexes to control access to the shared buffer and ensure proper synchronization between the Producer and Consumer.

2. Reader-Writer Problem

- The Reader-Writer Problem involves multiple processes that need to read from and write to shared data.
- Here, we have two types of processes: Readers, which only read the data, and Writers, which can modify the data.

Readers: These processes only read data from the shared resource and do not modify it.

Writers: These processes modify or write data to the shared resource.

- **The challenge** in the Reader-Writer problem is to allow multiple readers to access the shared data simultaneously without causing issues.
- However, only one writer should be allowed to write at a time, and no reader should be allowed to read while a writer is writing. This ensures the integrity and consistency of the data.

2. Reader-Writer Problem

Solution to Reader-Writer Problem

There are two fundamental solutions to the Readers-Writers problem:

- **Readers Preference:** In this solution, readers are given preference over writers. That means that till readers are reading, writers will have to wait. The Writers can access the resource only when no reader is accessing it.
- **Writer's Preference:** Preference is given to the writers. It simply means that, after arrival, the writers can go ahead with their operations, though perhaps there are readers currently accessing the resource.

3. Dining Philosophers Problem

- The Dining Philosopher Problem is a classic synchronization and concurrency problem that deals with resource sharing, deadlock, and starvation in systems where multiple processes require limited resources.

1. The Dining Philosopher Problem involves ‘n’ philosophers sitting around a circular table.
2. Each philosopher alternates between two states: thinking and eating.
3. To eat, a philosopher needs two chopsticks, one on their left and one on their right.
4. However, the number of chopsticks is equal to the number of philosophers, and each chopstick is shared between two neighboring philosophers.

The standard problem considers the value of ‘n’ as 5 i.e. we deal with 5 Philosophers sitting around a circular table.

3. Dining Philosophers Problem

Solution to Dining Philosophers Problem

- Solutions to this problem typically involve using synchronization techniques like semaphores or mutexes to ensure that the philosophers don't deadlock.
- One common approach is to use a monitor to coordinate access to the forks, allowing each philosopher to pick up and put down forks in a way that prevents deadlock.

Inter Process Communication (IPC)

- Processes need to communicate with each other in many situations.
- Inter-Process Communication or IPC is a mechanism that allows processes to communicate.
- It helps processes synchronize their activities, share information, and avoid conflicts while accessing shared resources.

Inter Process Communication (IPC)

Types of Process

1. **Independent process:** An independent process is not affected by the execution of other processes. Independent processes do not share any data or resources with other processes. No inter-process communication is required in this case.

2. **Co-operating process:** Interact with each other and share data or resources. A co-operating process can be affected by other executing processes. Inter-process communication (IPC) is a mechanism that allows processes to communicate with each other and synchronize their actions. The communication between these processes can be seen as a method of cooperation between them.

Fundamental models of Inter Process Communication (IPC)

- Inter process communication (IPC) allows different processes running on a computer to share information with each other.
- IPC allows processes to communicate by using different techniques like sharing memory, sending messages or using files.
- It ensures that processes can work together without interfering with each other.
- Cooperating processes require an Inter Process Communication (IPC) mechanism that will allow them to exchange data and information.

The **two fundamental models** of Inter Process Communication are:

1. **Shared Memory**
2. **Message Passing**

1. Shared Memory

- Communication between processes using shared memory requires processes to share some variable and it completely depends on how the programmer will implement it.
- Suppose process 1 and process 2 are executing simultaneously and they share some resources or use some information from another process.
- Process1 generates information about certain computations or resources being used and keeps it as a record in shared memory.
- When process 2 needs to use the shared information, it will check in the record stored in shared memory and take note of the information generated by process 1 and act accordingly.
- Processes can use shared memory for extracting information as a record from another process as well as for delivering any specific information to other processes.

2. Shared Memory

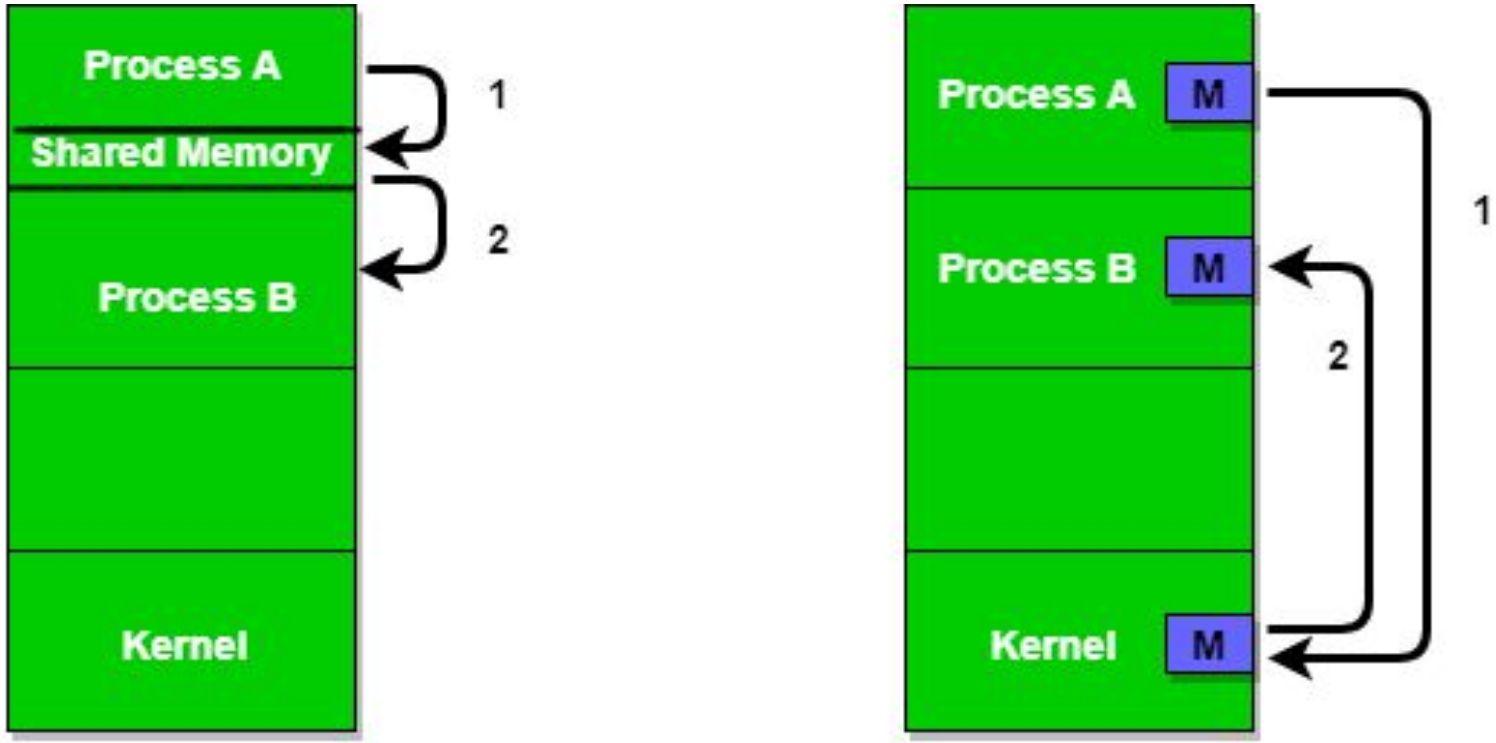


Figure 1 - Shared Memory and Message Passing

2. Message Passing

1. n message-passing systems, processes communicate with one another by sending and receiving messages over a communication channel.
2. The pattern of the connection provided by the channel is described by some topology systems.
3. The collection of the channels are called a network.
4. So by the definition of distributed systems, we know that they are geographically set of computers. So it is not possible for one computer to directly connect with some other node.
5. So all channels in the Message-Passing Model are private.
6. The sender decides what data has to be sent over the network. An example is, making a phone call.

2. Message Passing

7. The data is only fully communicated after the destination worker decides to receive the data. Example when another person receives your call and starts to reply to you.

8. There is no time barrier. It is in the hand of a receiver after how many rings he receives your call. He can make you wait forever by not picking up the call.

9. For successful network communication, it needs active participation from both sides.

Shared memory Deadlocks: Conditions

What is Deadlock?

- Deadlock is a situation in computing where **two or more processes are unable to proceed because each is waiting** for the other to release resources.
- Key concepts include **mutual exclusion, resource holding, circular wait, and no preemption.**
- Consider a practical **example** when two trains are coming toward each other on the same track and there is only one track, none of the trains can move once they are in front of each other.

Shared memory Deadlocks: Conditions

What is Deadlock?

- Deadlock is an **infinite Process it means that once a process goes into deadlock it will never come out of the loop** and the process will enter for an indefinite amount of time.
- There are only **detection, resolution, and prevention techniques. But, there are no Deadlock-stopping** techniques.

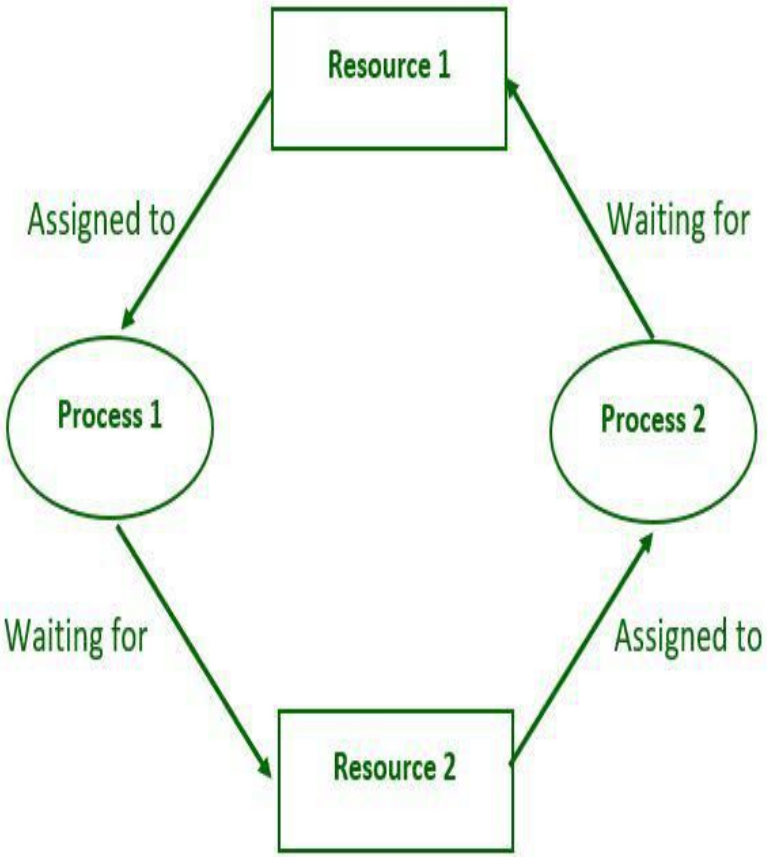


Figure: Deadlock in Operating system

Necessary Conditions for the Occurrence of a Deadlock

1. Mutual Exclusion
2. Hold and Wait
3. No Pre Emption
4. Circular Wait

Necessary Conditions for the Occurrence of a Deadlock

1. Mutual Exclusion

- Mutual Exclusion condition requires that **at least one resource be held in a non-shareable mode, which means that only one process can use the resource at any given time**.
- Both Resource 1 and Resource 2 are non-shareable in our scenario, and only one process can have exclusive access to each resource at any given time.

As an example:

- Process 1 obtains Resource 1.
- Process 2 acquires Resource 2.

Necessary Conditions for the Occurrence of a Deadlock

2. Hold and Wait

- The hold and wait condition specifies that a process **must be holding at least one resource while waiting for other processes to release resources** that are currently held by other processes.

Example,

- Process 1 has Resource 1 and is awaiting Resource 2.
- Process 2 currently has Resource 2 and is awaiting Resource 1.
- Both processes hold one resource while waiting for the other, **satisfying the hold and wait condition.**

Necessary Conditions for the Occurrence of a Deadlock

3. No Preemption

- Preemption is the **act of taking a resource from a process before it has finished its task. According to the no preemption condition, resources cannot be taken forcibly from a process a process** can only release resources voluntarily after completing its task.
-
- For example – Process p1 have resource r1 and requesting for r2 that is hold by process p2. then process p1 can not preempt resource r2 until process p2 can finish his execution. After some time it try to restart by requesting both r1 and r2 resources.

Problem – This can cause the Live Lock Problem .

Necessary Conditions for the Occurrence of a Deadlock

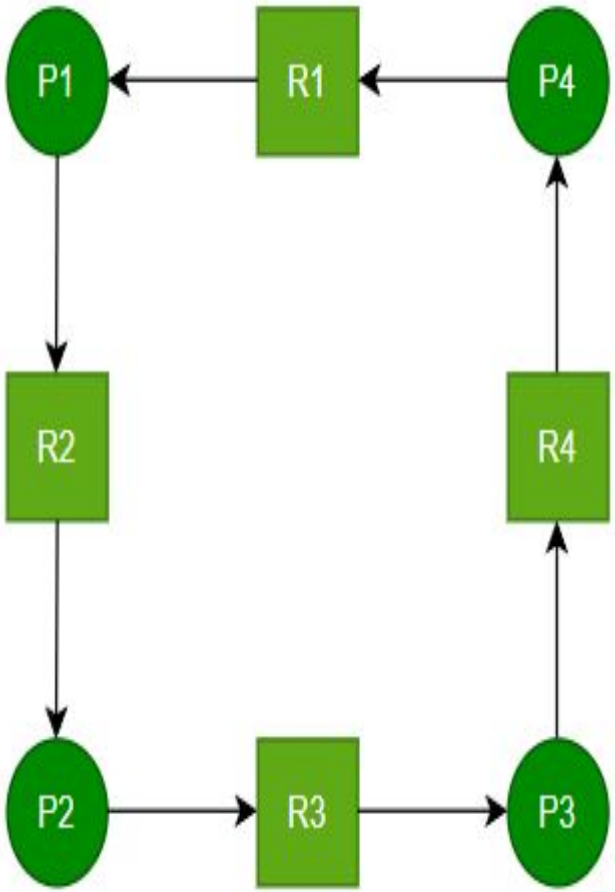
What is Live Lock?

- Live lock is the situation where **two or more processes continuously changing their state in response to each other without making any real progress.**
- Example:Suppose there are two processes 1 and 2 and two resources r1 and r2.
- Now, **p1 acquired r1 and need r2 & p2 acquired r2 and need r1.**
- so according to above method- Both p1 and p2 detect that they can't acquire second resource, so they release resource that they are holding and then try again.
- Continuous cycle- **p1 again acquired r1 and requesting to r2 p2 again acquired r2 and requesting to r1 so there is no overall progress still process are changing there state as they preempt resources and then again holding them. This the situation of Live Lock.**

Necessary Conditions for the Occurrence of a Deadlock

4. Circular Wait

- Circular wait is a condition in which a **set of processes are waiting for resources in such a way that there is a circular chain, with each process in the chain holding a resource that the next process needs.** This is one of the necessary conditions for a deadlock to occur in a system.
- **Example:** Imagine four processes—P1, P2, P3, and P4—and four resources—R1, R2, R3, and R4.



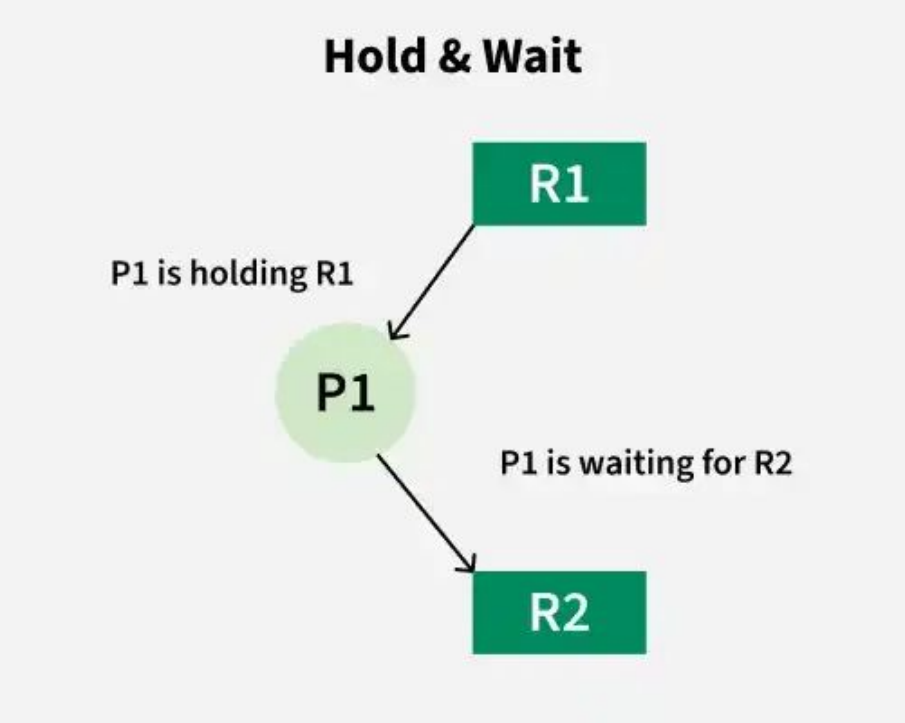
- P1 is holding R1 and waiting for R2 (which is held by P2).
- P2 is holding R2 and waiting for R3 (which is held by P3).
- P3 is holding R3 and waiting for R4 (which is held by P4).
- P4 is holding R4 and waiting for R1 (which is held by P1).

Deadlock Prevention

- We can prevent a Deadlock by eliminating any of the above four conditions.
- **Eliminate Mutual Exclusion**
- It is not possible to violate mutual exclusion because some resources, such as the **tape drive, are inherently non-shareable. For other resources, like printers, we can use a technique called Spooling** (Simultaneous Peripheral Operations Online).
- In spooling, when multiple processes request the printer, their jobs **(instructions of the processes that require printer access)** are added to the queue in the spooler directory.
- The printer is allocated to jobs on a **First-Come, First-Served (FCFS) basis. In this way, a process does not have to wait for the printer and can continue** its work after adding its job to the queue.

Deadlock Prevention

- **Eliminate Hold and Wait**
- Hold and wait is a condition in which a process holds one resource while simultaneously waiting for another resource that is being held by a different process. The process cannot continue until it gets all the required resources.



Deadlock Prevention

There are two ways to eliminate hold and wait:

- **By eliminating wait:** The process specifies the resources it requires in advance so that it does not have to wait for allocation after execution starts.

  For Example, Process1 declares in advance that it requires both Resource1 and Resource2.

- **By eliminating hold:** The process has to release all resources it is currently holding before making a new request.

  For Example: Process1 must release Resource2 and Resource3 before requesting Resource1.

Deadlock Prevention

There are two ways to eliminate hold and wait:

- **By eliminating wait:** The process specifies the resources it requires in advance so that it does not have to wait for allocation after execution starts.

  For Example, Process1 declares in advance that it requires both Resource 1 and Resource2.

- **By eliminating hold:** The process has to release all resources it is currently holding before making a new request.

  For Example: Process1 must release Resource2 and Resource3 before requesting Resource 1.

Deadlock Prevention

Eliminate Circular Wait

- To eliminate circular wait for deadlock prevention, we can use order on resource allocation.
- Assign a unique number to each resource.
- Processes can only request resources in an increasing order of their numbers.
- This prevents circular chains of processes waiting for resources, as no process can request a resource lower than what it already holds.

Detection and Recovery

- Another approach to dealing with deadlocks is to detect and recover from them when they occur.
- This can involve killing one or more of the processes involved in the deadlock or releasing some of the resources they hold.

Deadlock Prevention

Eliminate Circular Wait

- To eliminate circular wait for deadlock prevention, we can use order on resource allocation.
- Assign a unique number to each resource.
- Processes can only request resources in an increasing order of their numbers.
- This prevents circular chains of processes waiting for resources, as no process can request a resource lower than what it already holds.

Detection and Recovery

- Another approach to dealing with deadlocks is to detect and recover from them when they occur.
- This can involve killing one or more of the processes involved in the deadlock or releasing some of the resources they hold.

Deadlock Avoidance (Banker's Algorithm)

- Banker's Algorithm is a **resource allocation and deadlock avoidance algorithm used in operating systems.**
- It **ensures that a system remains in a safe state by** carefully allocating resources to processes while avoiding unsafe states that could lead to deadlocks.
- The Banker's Algorithm **is a smart way for computer systems to manage how programs use resources**, like memory or CPU time.
- It helps **prevent situations where programs get stuck and can not finish their tasks.** This condition is known as deadlock.
- By keeping track of what **resources each program needs and what's available, the banker algorithm makes sure** that programs only get what they need in a safe order.

Deadlock Avoidance (Banker's Algorithm)

- **Why Banker's Algorithm is Named So?**
- The banker's algorithm is named so because it is used in the banking system to **check whether a loan can be sanctioned** to a person or not.
- Suppose there are n number of account holders in a bank and the total sum of their money is S.
- Let us assume that the bank has a certain amount of money Y.
- If a **person applies for a loan then the bank first subtracts the loan amount from the total money that the bank has (Y) and if the remaining amount is greater than S** then only the loan is sanctioned.
- It is done because if all the a**ccount holders come to withdraw their money then the bank can easily** do it.

Deadlock Avoidance (Banker's Algorithm)

- **Why Banker's Algorithm is Named So? ...**
- It also helps the **OS to successfully share the resources between all the processes.**
- The banker's algorithm uses the notation of a **safe allocation state to ensure that granting a resource request c**annot lead to a deadlock either immediately or in the future.
- In other words, the bank would never allocate its money in such a way that it can no longer satisfy the needs of all its customers.
- The **bank would try to be in a safe state always.**

Components of the Banker's Algorithm

- Let **'n' be the number of processes** in the system and **'m' be the number of resource** types.

1. Available

- It is a 1-D array of size **'m'** indicating the number of available resources of each type.
- **Available[j] = k** means there are 'k' instances of resource type Rj

2. Max

- It is a 2-d array of size **'n*m'** that defines the maximum demand of each process in a system.
- **Max[i, j] = k** means process Pi may request at most 'k' instances of resource type Rj.

Components of the Banker's Algorithm

3. Allocation

- It is a 2-d array of **size 'n*m'** that defines the number of resources of each type currently allocated to each process.
- Allocation[i, j] = k means process Pi is currently allocated 'k' instances of resource type Rj

4. Need

- It is a 2-d array of size 'n*m' that indicates the remaining resource need of each process.
- Need [i, j] = k means process Pi currently needs 'k' instances of resource type Rj
- Need [i, j] = Max [i, j] – Allocation [i, j]

Banker's Algorithm

1. Active := *Running U Blocked*

for k = 1 to r:

New_request[k] := Requested_resources[requesting_process, k]

2. Simulated_allocation := *Allocated_resources*

for k = 1 to r: // Compute projected allocation state

Simulated_allocation[requesting_process, k] :=

Simulated_allocation[requesting_process, k] + New_request[k]

3. Feasible := *true*

for k = 1 to r: // Check if projected allocation state is feasible

if Total_resources[k] < Simulated_total_alloc[k]:

Feasible := false

Banker's Algorithm

4. if Feasible = true:

while set Active contains a process P1 such that:
for all k, Total_resources[k] - Simulated_total_alloc[k] >=
Max_need[P1, k] - Simulated_allocation[P1, k]:
Delete P1 from Active
for k = 1 to r:
Simulated_total_alloc[k] :=
Simulated_total_alloc[k] - Simulated_allocation[P1, k]

5. if set Active is empty:

for k = 1 to r: // Delete the request from pending requests
Requested_resources[requesting_process, k] := 0
for k = 1 to r: // Grant the request
Allocated_resources[requesting_process, k] :=
Allocated_resources[requesting_process, k] + New_request[k]
Total_alloc[k] := Total_alloc[k] + New_request[k]

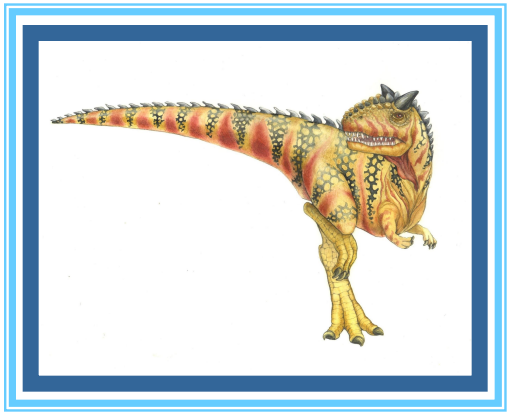
Banker's Algorithm Example

Key Concepts in Banker's Algorithm

Safe State: *There exists at least one sequence of processes such that each process can obtain the needed resources, complete its execution, release its resources, and thus allow other processes to eventually complete without entering a deadlock.*

Unsafe State: *Even though the system can still allocate resources to some processes, there is no guarantee that all processes can finish without potentially causing a deadlock.*

End



Silberschatz, Galvin and Gagne ©2018